PORTFOLIO OF EVIDENCE

SmartX

Implementation report. Simulated IoT gateway for hydroponics, utilities, and site-meter telemetry.

Alton Kholwani Muganda
Student number ST10263456
Module: PROG7312
POE

Contents

1. Executive summary 3
2. Project arc — Part 1 to Final PoE 4
2.1 Part 1 — Telemetry 4
2.2 Part 2 — Command stream 4
2.3 Part 3 — Mesh routing 5
3. Architecture of the unified system 6
4. How to demonstrate the completed system 7
5. Feedback-driven improvements 8
6. Task 3 — architectural and algorithmic challenges 9
6.1 Shared contracts 9
6.2 AVL history tree 9
6.3 Modelling the mesh 10
6.4 Traversal semantics 10
6.5 Prim’s MST 10
6.6 Keeping Parts 1 and 2 alive 11
7. Key learnings 12
7.1 Data structures 12
7.2 Algorithms 12
7.3 Async UI and background work 13
7.4 Integration discipline 13
7.5 A larger deployment 13
8. Recommended additional technologies 14
8.1 Cloud-hosted database 14
8.2 GitHub Actions 14
8.3 LLM-assisted alert summaries 14
8.4 Home Assistant 15
8.5 Containerisation 15
8.6 Observability 15
9. Conclusion 17
Appendix A — Key source map 18
Appendix B — Rendering this report 19
Bibliography 20

1. Executive summary

Smart-X is a simulated South African IoT gateway for hydroponics, utilities, and site-meter telemetry. The simulated devices are ESP32-class endpoints (Espressif Systems, n.d.). Heterogeneous packets — environmental floats, power integers, and actuator booleans — are validated and stored by an ASP.NET Core API (Microsoft, 2025i) and investigated in a Blazor WebAssembly operator console (Microsoft, 2025b).

The Final PoE delivers a unified three-pillar application. One `GatewayStore` singleton, registered for the process lifetime (Microsoft, 2025c), and one boot sequence seed the fleet, faults, queues, and mesh so a single API restart demos the full arc.

Pillar	Focus	Operator entry
Part 1	Sensor registration, typed ingest, Progressive Disclosure, health and freshness	`/telemetry`
Part 2	Real-time command stream: queues, undo stack, timeline, HashSet alerts, suggestions	`/commands`
Part 3	AVL history trees, mesh graph, BFS reachability, Prim MST, topology dashboard	`/mesh`

2. Project arc — Part 1 to Final PoE

2.1 Part 1 — Available Sensor Data Ingestion and Telemetry

Goal. Ingest heterogeneous telemetry and let operators investigate the fleet through Progressive Disclosure — overview, then filters, then device detail, then history — without hiding silent or faulty devices. Progressive disclosure keeps secondary detail behind an explicit next step so the first screen stays readable (Nielsen, 2006).

Delivered:

- Nested deployment forest (site, zone, sub-zone, node) with recursive validation. A tree is the natural model when each location contains further locations (Cormen et al., 2022).
- Registration and typed ingest for environmental, power, and actuator packets, held in `List<T>` collections (Microsoft, 2025g).
- Health and freshness classifiers (live, aging, stale, disconnected).
- Seeded fault scenarios: temperature spike, stuck actuator, silent sensor, malformed frames, location outage, and recovery.
- Multipart diagnostic attachments.
- Blazor investigation UI at levels L0 to L3 (Microsoft, 2025b).

Structures emphasised. Lists, dictionaries for device resolve, and recursive tree validation for deployment. The .NET `Dictionary<TKey,TValue>` is a hash table, so a successful lookup does not scan the fleet (Microsoft, 2025d). Hash-table dictionaries are expected constant time under the usual uniform-hashing assumptions (Cormen et al., 2022).

2.2 Part 2 — Real-Time Command Stream and History

Goal. Process critical alerts ahead of the FIFO backlog, support manual actuator overrides with undo, track unique disconnect and error keys, and surface predictive suggestions.

Delivered:

- A standard FIFO ingest queue, using `Queue<T>` (Microsoft, 2025l), beside a critical priority bypass queue. `PriorityQueue<TElement,TPriority>` dequeues the minimum priority key first (Microsoft, 2025k). Smart-X's key is severity, then earlier `RaisedAt`.
- A background `TelemetryQueueConsumer`, an ASP.NET Core hosted `BackgroundService` (Microsoft, 2026c). The request path only enqueues.
- Priority severity ranking with a `RaisedAt` tie-breaker.
- A command undo stack, `Stack<T>` (Microsoft, 2025n), and a `SortedDictionary` timeline so entries enumerate in key order without a separate sort on each read (Microsoft, 2025m).
- `HashSet` unique alert keys for disconnects and errors, so a duplicate key is rejected (Microsoft, 2025e).
- A recommendation engine with confidence scores normalised to the unit interval.
- A `/commands` console with filters, issue and undo, suggestions, and a five-second auto-refresh driven by `PeriodicTimer` (Microsoft, 2025j).

Lecturer feedback on Part 2 drove the Final PoE Stage 1 fixes: session state across navigation, a pillar-grouped menu, auto-refresh, the background consumer, the tie-breaker, disconnect detection, and confidence-based suggestions. The ledger is `docs/FEEDBACK-UPDATES.md`.

2.3 Part 3 — Network Topology and Mesh Routing

Goal. Model the radio mesh, query historical telemetry efficiently, traverse for reachability and healing, and compute a power-efficient routing backbone to the gateway. A wireless mesh is a multi-hop network in which nodes forward for one another and extra links provide resilience (Akyildiz, Wang and Wang, 2005). Smart-X simulates that shape. It does not implement an 802.11s or 802.15.4 stack.

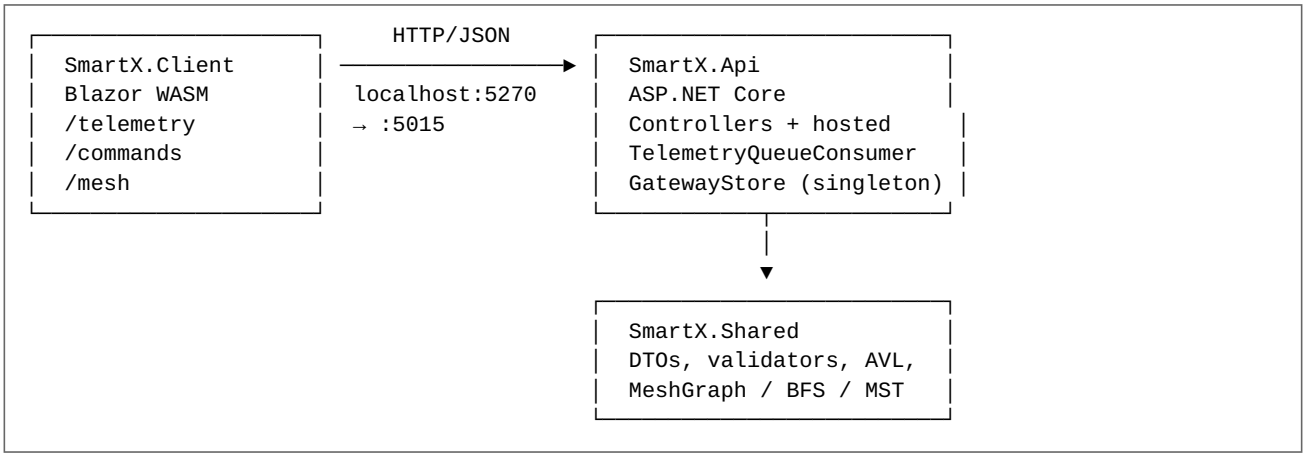
Delivered:

Structure or algorithm	Role	Evidence
AVL trees (per device)	Timestamp range history without a full list scan, about O(log n + k)	`AvlTelemetryHistoryTree`, `GET …/history/range`, `/mesh` historical log
Undirected mesh graph	Gateway plus 16 ESP32s, at least 20 weighted wireless links	`MeshGraph`, `MeshTopologySeeder`, `GET /api/mesh/topology`
BFS and DFS	Live reachability, down versus cut-off, alternate hops	`MeshGraphTraverser`, `MeshReachabilityAnalyzer`, `GET /api/mesh/traversal`
Prim’s MST	Minimum-cost backbone and parent hops toward the gateway	`MeshMstBuilder`, `GET /api/mesh/mst` (about 25 links down to 16, about 47% cost saved)
Topology UI	SVG mesh, route track by id or MAC, operator briefing, AVL log	`/mesh`

An AVL tree is a height-balanced binary search tree: after each insertion the heights of the two subtrees of every node differ by at most one (Adelson-Velsky and Landis, 1962). On a binary search tree, search and range walks cost time proportional to the height, and balance keeps that height logarithmic in the number of nodes (Cormen et al., 2022). Breadth-first and depth-first search are the standard graph walks; breadth-first search yields hop distance because it expands level by level (Cormen et al., 2022; Sedgewick and Wayne, 2011). Prim’s algorithm grows a minimum spanning tree from a chosen root by repeatedly adding the cheapest edge that leaves the tree (Prim, 1957).

Seeded mesh fault. `MeshDownScenarioSeeder` forces `sx-pwr-inverter` offline so healing hops are visible alongside the silent sump disconnect from Parts 1 and 2.

3. Architecture of the unified system



The Blazor WebAssembly client calls the API with `HttpClient` (Microsoft, 2026d). API and client agree on camelCase property names and string enums through `System.Text.Json` and `JsonStringEnumConverter` (Microsoft, 2026g; Microsoft, 2025f). Cross-origin calls from `http://localhost:5270` to `http://localhost:5015` are allowed by the API's CORS policy (Microsoft, 2026f).

Optional Docker Compose runs the same host ports (Docker, n.d.-a). There is no database. The in-memory store is reset by restarting the API.

4. How to demonstrate the completed system

1. Start the API, then the client, with `dotnet run` (Microsoft, 2026e) or with `docker compose up --build` (Docker, n.d.-a).
2. Open `http://localhost:5270`. All three pillars report Available.
3. Part 1: `/telemetry` shows 16 devices and the exceptions. Open a critical device.
4. Part 2: `/commands` shows the FIFO peak, suggestions, unique keys, and issue or undo of an override.
5. Part 3: `/mesh` shows the red down nodes, the tracked `sx-env-rack1-moist` MST path, the operator briefing (mesh versus MST savings), and the AVL historical log.

API smoke:

```
curl http://localhost:5015/api/sensors/summary
curl http://localhost:5015/api/stream/snapshot
curl http://localhost:5015/api/mesh/topology
curl http://localhost:5015/api/mesh/traversal
curl http://localhost:5015/api/mesh/mst
```

5. Feedback-driven improvements (Parts 1–2 → Final PoE)

Part 2 critique items were closed before and during the Part 3 work and logged in `docs/FEEDBACK-UPDATES.md`. They remain active in the unified app.

- Operator session state survives navigation because the Blazor filters live in a singleton service, not in fields that the router discards with the page (Microsoft, 2025c).
- Navigation is grouped by pillar rather than presented as a flat page list (Nielsen, 2006).
- The command stream auto-refreshes on a `PeriodicTimer` (Microsoft, 2025j).
- A background consumer drains the queues. Ingest does not drain them on the request thread (Microsoft, 2026c).
- The priority queue breaks ties on time, using the comparison key of `PriorityQueue<TElement,TPriority>` (Microsoft, 2025k).
- Disconnects are detected from freshness, then stored as `HashSet` keys so a repeat does not raise a second alert (Microsoft, 2025e).
- Suggestions carry a normalised confidence score. The moisture-to-pump rule is not hard-coded.

These behaviours support the Final PoE requirement that the earlier parts still function.

6. Task 3 — architectural and algorithmic challenges (and how they were overcome)

Final PoE Task 3 (Network Topology and Mesh Routing) was not a greenfield microservice. It had to land inside the existing Part 1 and Part 2 gateway without breaking Progressive Disclosure or the command stream (Nielsen, 2006). The challenges below are specific to this codebase.

6.1 Shared contracts across API and Blazor WASM

Challenge. The Blazor client must display topology, traversal, MST, and AVL range results without re-implementing graph logic in JavaScript. Enums such as `FreshnessState` and `MeshNodeKind`, and nested DTOs, break easily if the JSON shape drifts (Microsoft, 2026g).

Overcome.

- Mesh and AVL models and DTOs live in `SmartX.Shared`, referenced by both the API and the client.
- API and client agree on camelCase names and string enums (`JsonStringEnumConverter`) (Microsoft, 2025f).
- The client uses one `HttpClient`, `ApiBaseUrl` in `wwwroot/appsettings.json`, and `GatewayJson.Options` to deserialise (Microsoft, 2026d).
- CORS allows `http://localhost:5270` to call `http://localhost:5015`, including the Docker configuration in `appsettings.Docker.json` (Microsoft, 2026f).

Evidence. `MeshController` returns `MeshTopologySnapshotResponse`, `MeshTraversalResponse`, and `MeshMstResponse`. The `/mesh` page binds those same types.

6.2 Choosing and integrating an AVL history tree (not a list scan)

Challenge. Part 1 already stored packets in lists for Progressive Disclosure history (Microsoft, 2025g). Task 3 required range queries, for example moisture for one device between two timestamps, that stay efficient as history grows. A full list scan would fail the “trees for historical queries” criterion and would look identical to Part 1.

Overcome.

- A per-device AVL tree, `AvlTelemetryHistoryTree`, is keyed by timestamp then sequence and indexed by `TelemetryHistoryIndex` (Adelson-Velsky and Landis, 1962).
- Every successful ingest also inserts into the tree, on the same write path as list history.
- `QueryRange` prunes subtrees outside `[from, to]`. On a binary search tree a range walk only enters a subtree that can intersect the interval, so the cost follows the height plus the number of matches (Cormen et al., 2022). The API returns `nodesVisited` and `treeSize` so the UI can show that behaviour.
- The `/mesh` historical log and `GET /api/sensors/{id}/history/range` share that path.

Evidence. The demo device is `sx-env-rack1-moist` with metric `Moisture`. The range payload shows the match count and the visit count.

6.3 Modelling a realistic mesh on top of an existing fleet

Challenge. The graph could not be a toy triangle. It had to use the seeded 16 ESP32s (Espressif Systems, n.d.), include a gateway vertex, give nodes more than one neighbour, and give edges a non-negative weight that means something for the MST (a power or RSSI-style cost), while staying consistent with live freshness.

Overcome.

- `MeshGraph` is an adjacency list: nodes are the gateway plus the sensors, and edges are undirected and weighted. Adjacency lists are the usual representation when algorithms walk neighbours rather than scan a matrix (Sedgewick and Wayne, 2011).
- `MeshTopologySeeder` builds at least 20 links: rack clusters, the feeder bay, and an inter-rack backbone. Extra links are what give a mesh its alternate paths (Akyildiz, Wang and Wang, 2005).
- `RefreshOnlineStatus` reclassifies nodes from `LastSeenAt`, so Part 1 and Part 2 disconnects, such as the silent sump, appear as mesh-down.
- `MeshDownScenarioSeeder` ages `sx-pwr-inverter` so BFS healing is demoable after every boot.

Evidence. `GET /api/mesh/topology` returns 17 nodes and 25 edges. The offline count includes the inverter and the sump.

6.4 Traversal semantics: topology vs live routing

Challenge. A single BFS over every edge cannot answer both “what was the ideal path if everything were up?” and “who can still forward now?”. Operators also need an alternate hop when a relay is down, not only a visit-order list (Cormen et al., 2022).

Overcome.

- `MeshGraphTraverser` supports BFS and DFS, with an `onlineOnly` flag that does not expand offline neighbours (Sedgewick and Wayne, 2011).
- `MeshReachabilityAnalyzer` runs a topology BFS and a live BFS, then classifies each device as reachable, down, or cut-off. Cut-off means the device is online but stranded behind a failed relay.
- For each down node it suggests a live neighbour closest, by hop count, to the gateway, and it exposes the ideal path beside the alternate.
- The API is `GET /api/mesh/traversal`. The UI is the operator briefing on `/mesh`. The HTTP traversal reports BFS, because hop distance is what routing uses. DFS remains available on `MeshGraphTraverser.DepthFirst`.

Evidence. Traversal lists `sx-pwr-inverter` and `sx-env-sump-temp` as down, with suggested next hops, typically `sx-gateway`.

6.5 MST choice (Prim) and comparing it to the full mesh

Challenge. The rubric accepts Prim or Kruskal. The product need is a gateway-rooted routing tree with parent hops for telemetry, plus a clear comparison of the full mesh with the efficient backbone.

Overcome.

- `MeshMstBuilder` implements Prim's algorithm (Prim, 1957). Kruskal's algorithm also builds a minimum spanning tree, by sorting edges and unioning components (Kruskal, 1956). Prim was chosen because the tree is rooted at the gateway, the graph is already an adjacency list, and the parent of each node falls out of the growth from that root (Sedgewick and Wayne, 2011).
- `onlineOnly=true` drives route tracking on `/mesh`. The full MST payload includes `MeshVsMstComparison`: edges saved, cost saved, and the redundant links.
- The seeded comparison used in demos is stable: about 25 links at cost 42.7, against 16 links at cost 22.7, about 47 percent of the radio cost saved.

Evidence. `GET /api/mesh/mst` returns the comparison block. The SVG highlights `sx-env-rack1-moist` through to `sx-gateway`.

6.6 Keeping Part 1 and Part 2 alive under Part 3 load

Challenge. Mesh APIs and a heavy `/mesh` page risked regressions: starving the queue consumer, blocking the UI, or treating Part 3 as a separate app.

Overcome.

- Mesh reads refresh status under the existing `GatewayStore` lock. They do not replace the ingest and queue paths.
- The hosted `TelemetryQueueConsumer` keeps draining priority then FIFO, independently of topology GETs (Microsoft, 2026c).
- `/mesh` loads topology, MST, traversal, and history asynchronously, with explicit callouts when the API is down, empty, or in error (Microsoft, 2025b).
- The startup menu and the sidebar keep the three pillars. The README documents a one-boot demo across all three.

Evidence. A fresh-restart smoke (checklist Stage 9.3) shows the summary, the stream snapshot and suggestions, and the mesh topology, traversal, and MST all succeeding together.

7. Key learnings (linked to actual Smart-X tasks)

These lessons come from implementing Parts 1 to 3 in one gateway. They are not a textbook summary detached from the code.

7.1 Data structures — pick the structure that matches the operator question

Operator question	Structure used in Smart-X	Why it fits
Which device is this id or MAC?	Dictionaries in `SensorFleet`	Expected O(1) resolve on ingest and command (Microsoft, 2025d; Cormen et al., 2022)
What happened to overrides and alerts over time?	`SortedDictionary` timeline	Ordered history without re-sorting a list on each GET (Microsoft, 2025m)
Has this disconnect already been raised?	`HashSet` unique keys	Duplicate suppression (Microsoft, 2025e)
Moisture for rack 1 between T1 and T2?	Per-device AVL tree	Range walk about log n + k; `nodesVisited` shows it (Adelson-Velsky and Landis, 1962)
Who is linked on the radio mesh?	Adjacency-list `MeshGraph`	Neighbour scans, BFS, and Prim (Sedgewick and Wayne, 2011)
Cheapest backbone to the gateway?	Prim MST parent map	Gateway-rooted hops for `/mesh` (Prim, 1957)
Critical alert versus FIFO backlog?	Priority queue plus FIFO queue	Severity, then time, versus arrival order (Microsoft, 2025k; Microsoft, 2025l)

Learning. Structures were chosen for the query the UI or API must answer, then seeded so the demo is deterministic. Replacing a list with an AVL paid off because ingest writes both and the range API exposes the efficiency counts the operator can see.

7.2 Algorithms — mesh theory applied to a hydroponics gateway

- **BFS versus DFS.** BFS from `sx-gateway` gives hop distance and healing paths. DFS remains available for a deep walk. Reachability reporting prefers BFS because the useful question is how many hops lie between a device and a live neighbour (Cormen et al., 2022).
- **Online-only walks.** IoT meshes fail in part, not only in whole (Akyildiz, Wang and Wang, 2005). Running a topology BFS beside a live BFS separates down (offline) from cut-off (online but stranded). Both appear in `GET /api/mesh/traversal`.
- **MST as a radio budget.** The full mesh, about 25 links, is the resilient graph. Prim's tree, about 16 links, is the day-to-day backbone (Prim, 1957). The comparison payload, about 47 percent of radio cost saved, makes that concrete.
- **Prim rather than Kruskal here.** Telemetry routes toward a fixed gateway, so growing the tree from that root yields parent hops without a second rooting pass (Kruskal, 1956; Sedgewick and Wayne, 2011).

Learning. Connectivity, faults, and spanning trees only show up clearly when they are tied to seeded devices (`sx-pwr-inverter` down, the moist-sensor path highlighted) and to plain wording on `/mesh`.

7.3 Async UI and background work — do not block the operator

- Part 2 feedback made the cost of a synchronous queue drain obvious. Ingest must enqueue only. `TelemetryQueueConsumer` drains priority, then FIFO (Microsoft, 2026c).
- `PeriodicTimer` on `/commands` (five seconds) refreshes KPIs and suggestions without a manual reload, and it pauses during override and undo so the operator's action is not raced (Microsoft, 2025j).
- `OperatorSessionState` is required because a Blazor "page" is a component. Filters kept only in the component disappear on navigation (Microsoft, 2025c).
- `/mesh` loads topology, traversal, MST, and AVL history asynchronously, with explicit callouts, so a dead port 5015 is obvious instead of an endless spinner (Microsoft, 2025b).

Learning. A functioning operator app stays usable while the gateway does the work. Async Blazor plus a hosted consumer separates operator latency from queue processing.

7.4 Integration discipline — one store, three pillars

- Sharing `GatewayStore` forced freshness, alerts, and mesh online flags to stay consistent. The silent sump and the mesh-down inverter are the same fact.
- `SmartX.Shared` is the contract layer, so graph and AVL logic are not duplicated in the browser (Microsoft, 2026d).
- Docker Compose later reused the same ports and the same CORS story. Containers did not require a second architecture (Docker, n.d.-a).

Learning. Evolving Part 1, then Part 2, then Part 3 in one process is harder than three demos. It matches the brief, and the repository tells one story.

7.5 What I would watch in a larger deployment

In-memory state and boot seeds are appropriate for PROG7312 marking. A production deployment would add durable storage, authentication, and horizontal scale, without abandoning the same structures: indexed history, a graph snapshot, and an MST recomputed when the topology changes. Those directions are outlined in Stages 8.7–8.8.

8. Recommended additional technologies

Smart-X already runs locally, and optionally under Docker Compose (Docker, n.d.-a). The tools below would extend it toward a production-style IoT operations platform. Each one is tied to a concrete gap in Parts 1 to 3, and each is judged on security, scalability, performance, and workflow inside the .NET ecosystem.

8.1 Cloud-hosted database (Azure SQL / Cosmos DB / PostgreSQL on Azure)

Replace. The in-memory `GatewayStore`, which is lost on every API restart.

Add. Persistent storage for sensors, packet history, commands, alerts, and mesh topology snapshots.

Why for Smart-X. Markers and demos reset cleanly today. A real hydroponics site needs history across reboots and across more than one API instance. AVL and range queries would then read from indexed time-series or document partitions instead of process RAM.

.NET rationale. EF Core and the Azure Cosmos DB SDK plug into the existing ASP.NET Core dependency-injection pipeline without leaving C# (Microsoft, 2020; Microsoft, 2026h). Azure SQL Database is the managed relational option on the same platform (Microsoft, 2026i). Security: managed identity and Azure Key Vault, so connection strings are not stored in `appsettings`, and database RBAC instead of a shared administrator password (Microsoft, 2025h; Microsoft, 2026b). Scalability: several API replicas share one store; a Cosmos partition key such as `sensorId` or site matches fleet growth (Microsoft, 2026h). Performance: indexed range queries replace linear in-memory scans for history windows, while hot AVL and mesh graphs can stay in memory with a durable write-behind. Workflow: the same `dotnet` tooling already used for the solution.

8.2 GitHub Actions CI/CD

Replace. A manual `dotnet build`, and "it works on my machine", before a push.

Add. Workflows on `push` and `pull_request` that restore, build the API and the client, optionally run smoke curls against a test host, and publish Docker images (GitHub, n.d.; Microsoft, 2026e).

Why for Smart-X. The `POE` branch already has a long commit history. CI would guard mesh and queue regressions when feedback fixes and Part 3 land together.

.NET rationale. Official setup and `dotnet restore`, `build`, `test`, and `publish` are first-class, and a static Blazor publish fits the same job matrix as the API (Microsoft, 2026e). Security: OIDC federation to a registry avoids long-lived deploy secrets. Scalability: parallel jobs for API and client, and a NuGet cache across runs. Performance: fail on compile or test before a Docker layer build spends minutes (Docker, n.d.-b). Workflow: pull-request checks on `POE` and `main` match the existing GitHub remote (`akzz98/SmartX`).

8.3 LLM-assisted alert summaries (Azure OpenAI / GitHub Models API)

Replace. Operators reading raw `HashSet` keys and long alert reasons on `/commands` (Microsoft, 2025e).

Add. A summarisation endpoint that turns recent priority alerts and disconnect keys into a short operator brief, for example irrigation risk on rack 1, with the inverter relay down and a gateway hop suggested.

Why for Smart-X. Part 2 suggestions already score patterns. A language-model layer would narrate the current incident without replacing the deterministic recommendation engine.

.NET rationale. Semantic Kernel can sit as a scoped service beside the existing controllers (Microsoft, 2023). Azure OpenAI, now surfaced in Microsoft's Foundry model docs, is the hosted model endpoint that brief would call (Microsoft, 2026j). Security: keep prompts on the server, do not send actuator secrets, and rate-limit the endpoint so the WebAssembly client cannot burn quota. Scalability: queue summary jobs, reusing the priority-queue pattern, so ingest latency stays untouched (Microsoft, 2025k). Performance: call the model only on an alert burst or an operator request, and cache the brief against a hash of the disconnect set. Workflow: a feature flag in `appsettings` toggles summaries without forking the Blazor UI.

8.4 Home Assistant (MQTT / REST integration)

Replace. Simulation-only ESP32 traffic into the gateway (Espressif Systems, n.d.).

Add. A bridge from Smart-X commands and telemetry to Home Assistant entities — valves, pumps, moisture sensors — over MQTT or the Home Assistant REST API (Home Assistant, n.d.-a; Home Assistant, n.d.-b).

Why for Smart-X. This matches real site automation. Part 2 overrides such as `OpenValve` and `StartPump` become physical or virtual actuators, while Progressive Disclosure still investigates gateway state (Nielsen, 2006).

.NET rationale. MQTTnet is a .NET MQTT client and server library, and `HttpClient` is already the factory pattern used by the Blazor client (MQTTnet, n.d.; Microsoft, 2026d). MQTT itself is the OASIS publish/subscribe protocol widely used for device telemetry (OASIS, 2019). A hosted `BackgroundService` can bridge Home Assistant and `GatewayStore` in the same way as today's queue consumer (Microsoft, 2026c). Security: long-lived tokens in a secret store, TLS on MQTT, and the same authorisation on override commands before anything is published. Scalability: MQTT decouples bursty device traffic from API threads, and sites can be separate topics. Performance: asynchronous publish and subscribe avoids blocking ingest. Workflow: a local Home Assistant plus the Docker Compose stack already used here can demo the bridge before any cloud IoT hub is added (Docker, n.d.-a).

8.5 Deeper containerisation / orchestration (already started)

Current. `docker compose up --build` for the API and an nginx WebAssembly client (Docker, n.d.-a; nginx, n.d.).

Extend. Azure Container Apps or Kubernetes for replicas, health probes, and separate scaling of the API and the static client, plus a volume or database for `App_Data` attachments (Microsoft, 2026a; Kubernetes, n.d.).

Why for Smart-X. Compose proves packaging. Orchestration matters once a cloud database and CI publish images continuously.

.NET rationale. The existing multi-stage Dockerfiles use the official SDK and ASP.NET runtime images (Docker, n.d.-b). Container Apps speaks the platform's health endpoints (Microsoft, 2026a). Security: non-root containers, a private registry, and managed identity to the database (Microsoft, 2025h). Scalability: scale API replicas independently of the static nginx client; scale on queue depth later. Performance: layer caching, and trimmed or ReadyToRun publish, for cold starts. Workflow: the same Compose files can graduate toward Container Apps or Kubernetes with little Dockerfile change, and CI can publish the images the workflows already build (GitHub, n.d.).

8.6 Observability (OpenTelemetry + Application Insights or Grafana)

Replace. Console logs and manual curl health checks.

Add. Traces for ingest, queue, and consumer; metrics for queue depth and MST recompute time; dashboards for the disconnect rate.

Why for Smart-X. Part 2 queue behaviour and Part 3 traversal cost become measurable in operations, not only in README curls.

.NET rationale. OpenTelemetry's .NET SDK, with ASP.NET Core instrumentation, is the usual way to export traces and metrics without rewriting controllers (OpenTelemetry, n.d.). Application Insights is one Azure Monitor exporter of that telemetry (Microsoft, 2025a). Security: scrub operator notes and attachment paths from spans, and restrict the dashboard. Scalability: sampling keeps high-ingest telemetry affordable; use metrics, not a full trace, for every packet. Performance: spans around BFS and MST show when those algorithms dominate CPU on a larger mesh (Cormen et al., 2022). Workflow: local counters first, then export to Azure Monitor without a second instrumentation pass.

9. Conclusion

Smart-X progressed from telemetry with Progressive Disclosure (Nielsen, 2006), through a real-time command and alert pipeline built on queues, a stack, a hash set, and a hosted consumer (Microsoft, 2025l; Microsoft, 2026c), to a network-topology pillar: an AVL history tree (Adelson-Velsky and Landis, 1962), BFS and DFS (Cormen et al., 2022), and a Prim backbone (Prim, 1957). The Final PoE artefact is not a separate app. It is the same gateway, with trees, graphs, and the MST wired into the APIs and the `/mesh` dashboard, and with the Part 2 lecturer feedback addressed in the feedback ledger.

Appendix A — Key source map

Concern	Location
Fleet, queues, and mesh store	`SmartX.Api/Storage/GatewayStore.cs`
Background consumer	`SmartX.Api/Storage/TelemetryQueueConsumer.cs`
Mesh seed and down scenario	`SmartX.Api/Simulation/MeshTopologySeeder.cs`, `MeshDownScenarioSeeder.cs`
Mesh APIs	`SmartX.Api/Controllers/MeshController.cs`
AVL history	`SmartX.Shared/Collections/AvlTelemetryHistoryTree.cs`
BFS and reachability	`SmartX.Shared/Mesh/MeshGraphTraverser.cs`, `MeshReachabilityAnalyzer.cs`
Prim MST	`SmartX.Shared/Mesh/MeshMstBuilder.cs`
Topology UI	`SmartX.Client/Pages/Mesh.razor`
Feedback ledger	`docs/FEEDBACK-UPDATES.md`
Operator how-to	`README.md`

Appendix B — Rendering this report

This file is the implementation report for the PoE. The repository does not contain a separate document titled “Implementation report”. The operator guide is `README.md`. The lecturer-feedback ledger is `docs/FEEDBACK-UPDATES.md`.

The PDF committed beside this file is rendered with Pandoc and WeasyPrint. From the repository root:

```
pandoc docs/PROJECT-COMPLETION-REPORT.md \
  --from markdown \
  --to html5 \
  --standalone \
  --embed-resources \
  --no-highlight \
  --metadata title="SmartX — Implementation report" \
  --css=docs/report.css \
  -o /tmp/smartx-report.html
weasyprint /tmp/smartx-report.html docs/PROJECT-COMPLETION-REPORT.pdf
```

`docs/report.css` sets the cover page, the contents leaders, A4 margins, page numbers, and rules that keep headings with the following text and keep tables and code blocks from splitting across pages.

Bibliography

Adelson-Velsky, G. and Landis, E. (1962) ‘An algorithm for the organization of information’, *Proceedings of the USSR Academy of Sciences*, 146, pp. 263–266. English translation by M.J. Ricci, *Soviet Mathematics Doklady*, 3, pp. 1259–1263.

Akyildiz, I.F., Wang, X. and Wang, W. (2005) ‘Wireless mesh networks: a survey’, *Computer Networks*, 47(4), pp. 445–487. doi: 10.1016/j.comnet.2004.12.001.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to algorithms*. 4th edn. Cambridge, MA: MIT Press.

Docker (n.d.-a) *Docker Compose*. Available at: https://docs.docker.com/compose/ (Accessed: 13 November 2026).

Docker (n.d.-b) *Multi-stage builds*. Available at: https://docs.docker.com/build/building/multi-stage/ (Accessed: 13 November 2026).

Espressif Systems (n.d.) *ESP32 Wi-Fi and Bluetooth SoC*. Available at: https://www.espressif.com/en/products/socs/esp32 (Accessed: 13 November 2026).

GitHub (n.d.) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions (Accessed: 13 November 2026).

Home Assistant (n.d.-a) *MQTT*. Available at: https://www.home-assistant.io/integrations/mqtt/ (Accessed: 13 November 2026).

Home Assistant (n.d.-b) *REST API*. Available at: https://contributors.home-assistant.io/core/internals/rest-api/ (Accessed: 13 November 2026).

Kruskal, J.B. (1956) ‘On the shortest spanning subtree of a graph and the traveling salesman problem’, *Proceedings of the American Mathematical Society*, 7(1), pp. 48–50. doi: 10.1090/S0002-9939-1956-0078686-7.

Kubernetes (n.d.) *Overview*. Available at: https://kubernetes.io/docs/concepts/overview/ (Accessed: 13 November 2026).

Microsoft (2020) *Overview of Entity Framework Core*. Available at: https://learn.microsoft.com/en-us/ef/core/ (Accessed: 13 November 2026).

Microsoft (2023) *Introduction to Semantic Kernel*. Available at: https://learn.microsoft.com/en-us/semantic-kernel/overview/ (Accessed: 13 November 2026).

Microsoft (2025a) *Application Insights OpenTelemetry observability overview*. Available at: https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview (Accessed: 13 November 2026).

Microsoft (2025b) *ASP.NET Core Blazor*. Available at: https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2025c) *ASP.NET Core Blazor dependency injection*. Available at: https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/dependency-injection?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2025d) *Dictionary<TKey,TValue> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025e) *HashSet<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.hashset-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025f) *JsonStringEnumConverter class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.text.json.serialization.jsonstringenumconverter?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025g) *List<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025h) *Managed identities for Azure resources*. Available at: https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview (Accessed: 13 November 2026).

Microsoft (2025i) *Overview of ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2025j) *PeriodicTimer class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.threading.periodictimer?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025k) *PriorityQueue<TElement,TPriority> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.priorityqueue-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025l) *Queue<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.queue-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025m) *SortedDictionary<TKey,TValue> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.sorteddictionary-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025n) *Stack<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.stack-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2026a) *Azure Container Apps overview*. Available at: https://learn.microsoft.com/en-us/azure/container-apps/overview (Accessed: 13 November 2026).

Microsoft (2026b) *Azure Key Vault overview*. Available at: https://learn.microsoft.com/en-us/azure/key-vault/general/overview (Accessed: 13 November 2026).

Microsoft (2026c) *Background tasks with hosted services in ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2026d) *Call a web API from an ASP.NET Core Blazor app*. Available at: https://learn.microsoft.com/en-us/aspnet/core/blazor/call-web-api?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2026e) *dotnet run command*. Available at: https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-run (Accessed: 13 November 2026).

Microsoft (2026f) *Enable cross-origin requests (CORS) in ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2026g) *How to customize property names and values with System.Text.Json*. Available at: https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/customize-properties (Accessed: 13 November 2026).

Microsoft (2026h) *Azure Cosmos DB overview*. Available at: https://learn.microsoft.com/en-us/azure/cosmos-db/overview (Accessed: 13 November 2026).

Microsoft (2026i) *What is the Azure SQL Database service?* Available at: https://learn.microsoft.com/en-us/azure/azure-sql/database/sql-database-paas-overview?view=azuresql (Accessed: 13 November 2026).

Microsoft (2026j) *What's new in Azure OpenAI in Microsoft Foundry Models (classic)*. Available at: https://learn.microsoft.com/en-us/azure/foundry-classic/openai/whats-new (Accessed: 13 November 2026).

MQTTnet (n.d.) *MQTTnet*. Available at: https://github.com/dotnet/MQTTnet (Accessed: 13 November 2026).

nginx (n.d.) *nginx documentation*. Available at: https://nginx.org/en/docs/ (Accessed: 13 November 2026).

Nielsen, J. (2006) ‘Progressive disclosure’, *Nielsen Norman Group*, 4 December (updated 15 July 2022). Available at: https://www.nngroup.com/articles/progressive-disclosure/ (Accessed: 13 November 2026).

OASIS (2019) *MQTT Version 5.0*. OASIS Standard. Available at: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html (Accessed: 13 November 2026).

OpenTelemetry (n.d.) *.NET*. Available at: https://opentelemetry.io/docs/languages/dotnet/ (Accessed: 13 November 2026).

Prim, R.C. (1957) ‘Shortest connection networks and some generalizations’, *Bell System Technical Journal*, 36(6), pp. 1389–1401. doi: 10.1002/j.1538-7305.1957.tb01515.x.

Sedgewick, R. and Wayne, K. (2011) *Algorithms*. 4th edn. Addison-Wesley.